

Topic: Backpropagation in CNN (Part 1)

(How CNNs actually learn from data)

Introduction

You already know CNNs have layers like:

- **Convolution layers** → detect features
- **Pooling layers** → reduce size and keep strong features
- **Fully connected layers** → make final predictions

But the real question is —

How do CNNs know what the right filter values (weights) should be?

That's where **Backpropagation** comes in.

Backpropagation in CNN

Backpropagation (short for “*backward propagation of errors*”) is the learning mechanism that updates all the **weights** and **biases** in the CNN after each prediction.

In simple terms:

1. CNN makes a prediction (Forward pass)
2. Compares it to the correct answer (Computes loss)
3. Calculates how wrong it was (Error)
4. Sends that error *backward* through the network
5. Adjusts filters and weights so that next time, the error decreases

So the main goal is:

Reduce the error by updating weights using the loss gradient.

Trainable Parameters

CNNs have **trainable parameters**, which means the network *learns* them automatically during training.

Let's identify what parameters CNN learns 📌

1 Filter weights

- These are the main parameters in convolution layers.
- Example: If you have 32 filters, each of size $3 \times 3 \times 3$,
→ Number of weights = $32 \times 3 \times 3 \times 3 = \mathbf{864 \text{ trainable parameters.}}$

Each of these filter weights is updated during backpropagation.

2 Bias terms

- Each filter also has one bias value.
 - So total biases = number of filters.
-

Together, **weights + biases** = total *trainable parameters* in convolution layers.

Note: Pooling layers **don't** have trainable parameters — they just perform an operation (like max or average pooling).

Logical Flow of Backpropagation

Let's understand the logical steps CNN follows during learning.

Step 1: Forward Propagation

CNN makes a prediction.

Step 2: Calculate Loss

Compare predicted output with actual target using a **loss function**, e.g.:

- For classification → *Categorical Cross Entropy*
- For regression → *Mean Squared Error (MSE)*

Step 3: Compute Gradients

Find how much each parameter (weight/bias) contributed to the error.

This is done by applying **the chain rule of calculus** (partial derivatives).

Step 4: Backward Propagation

Send the error backward:

- From **output layer** → **fully connected layer** → **convolution layer**
- Compute **gradients** for each layer's weights.

Step 5: Update Parameters

Use **Gradient Descent** to adjust weights:

$$W_{new} = W_{old} - \eta \times \frac{\partial L}{\partial W}$$

where

- η = learning rate
 - $\frac{\partial L}{\partial W}$ = gradient of loss with respect to weight
-

Forward Propagation (Detailed View)

Let's recap what happens in a CNN during the **forward pass**, since understanding this is key to understanding how errors flow backward later.

Step-by-step forward pass:

[1] Input Image → enters the CNN.

Example: 32×32×3 (RGB image).

[2] Convolution Layer

- Filters slide over the image.
-

- At each position → dot product (matrix multiplication).
- Result = **feature map** (shows where certain patterns appear).

Mathematically:

$$Y = X * W + b$$

where

- X = input
- W = filter (kernel)
- b = bias

3 Activation Function (ReLU)

- Applies non-linearity: $f(x) = \max(0, x)$
- Keeps positive signals, removes negatives → allows CNN to learn complex patterns.

4 Pooling Layer

- Reduces feature map size (down-sampling).
- Keeps strong features, removes noise.

5 Flattening Layer

- Converts 2D feature maps → 1D vector (to connect to dense layers).

6 Fully Connected (Dense) Layer

- Every neuron connects to all activations from the previous layer.
- Combines learned features to make final classification.

7 Output Layer

- Produces final probabilities (e.g., using Softmax).

The output is compared to the actual label → **Loss function** calculates how wrong it is.

Now, Backpropagation Begins

Once we have the loss:

- We calculate how the **loss changes with respect to each neuron's output**, then each neuron's **input**, then each **weight**.
- These are called **gradients**.

We move **backward layer by layer**:

- From **output layer** → **fully connected** → **convolution** → **input**.

Each layer sends “how much I contributed to the error” back to the previous layer.

What Happens Mathematically (Simplified)

For convolution layers:

$$\frac{\partial L}{\partial W} = X * \frac{\partial L}{\partial Y}$$

- $X \rightarrow$ input
- $Y \rightarrow$ output (feature map)
- $L \rightarrow$ loss

For fully connected layers:

$$\frac{\partial L}{\partial W} = \text{input}^T \times \frac{\partial L}{\partial \text{output}}$$

Then, we update all weights using **Gradient Descent**.

Outro (Summary)

Let's recap what we've learned

Concept	Meaning
Backpropagation	Process of sending error backward to update weights
Trainable Parameters	Filter weights + biases (learned automatically)
Forward Propagation	Input $\rightarrow$ Convolution $\rightarrow$ Pooling $\rightarrow$ Dense $\rightarrow$ Output
Loss Function	Measures how wrong the prediction is
Gradients	Show how much each weight contributed to the error
Weight Update Rule	($W_{\text{new}} = W_{\text{old}} - \eta \times \text{gradient}$)
Goal	Minimize loss by adjusting all parameters

In Simple Words:

CNNs **learn by making mistakes** then slowly correcting them using math (gradients).

Every training step helps filters “see better” — from random noise $\rightarrow$ clear object features.
